Lab 5

Jooho Oh

02-15-2023

Submission Instructions

This lab (and all labs) will be graded for correctness. To submit the lab:

1. Once all code is completed and saved, in the RStudio ribbon, click Knit > Knit to PDF.
2. If you have trouble knitting **anything** to PDF, please work with the Instructor or GSI to resolve the issue. It is okay to instead knit to HTML, then convert to PDF using Adobe or other software.
3. If there is any handwritten work for the lab, combine the handwritten work with the knitted PDF into a single PDF file.
4. Submit the PDF to Gradescope, **selecting** pages for questions. *You will lose half a point on the lab if you don't select pages.*

Lab 5 is due to Gradescope by Sunday, February 19th at 11:59PM PT.

Today's lab will work from our simulation tools in Lab 4 to simulate: * omitted variables bias * consider validation prediction performance as an alternative model evaluation tool.

Part I: Reposting relevant code from Lab 4

In this section I am pasting some of the useful simulation scripts that we wrote in lab 4 that will be useful to us. No need for any work here other than to briefly inspect what we are reusing.

```
generateX <- function(n,p) {
    data_vec <- rnorm(n*p)
    data_matrix_no_intercept <- matrix(data_vec, nrow = n, ncol = p)
    data_matrix <- cbind(rep(1, n), data_matrix_no_intercept)
    return(data_matrix)
}

generateBeta <- function(p) {
    beta_no_intercept <- pracma::linspace(0.5, -0.5, p)
    beta_1d <- c(0, beta_no_intercept)
    beta_vec <- matrix(beta_1d, ncol=1)
    return(beta_vec)
}


generateY <- function(X, beta, sigma = 1) {
    n <- dim(X)[1]
```

```
    eps <- matrix(rnorm(n, sd = sigma), ncol=1)

    Y <- X %*% beta + eps

    return(Y)
}

includeRandomCovars <- function(X, n_random_covars = 5) {
    n <- dim(X)[1]
    random_covars <- matrix(rnorm(n*n_random_covars), nrow=n)
    X_w_random <- cbind(X, random_covars)

    return(X_w_random)
}


collinearitySimulation <- function(n, p, n_correlated_covars = 1, corr = 0.6) {

    correlated_variates <-
        faux::rnorm_multi(n = n, vars = n_correlated_covars + 1, r = corr)

    ## main DGP:
    X <- generateX(n, p)

    ## replace first non-intercept covariate with first correlated_variate:
    X[,2] <- correlated_variates[,1]


    beta <- generateBeta(p)
    Y <- generateY(X, beta)

    X_w_correlated_variates <- cbind(X, correlated_variates[,2:dim(correlated_variates)[2]])

    return(list(X=X, X_w_correlated_variates=X_w_correlated_variates, beta=beta, Y=Y))
}

modelCorrData <- function(n, p, n_correlated_covars = 1, corr = 0.6, seed = 5) {
    ## we will set seed to make sure the results are reproducible
    set.seed(seed)

    sim2_data <- collinearitySimulation(n, p, n_correlated_covars = n_correlated_covars, corr = corr)

    ## get true beta1
    true_beta1 <- sim2_data$beta[2]

    ## runs model on the "true" data matrix
    true_model_res <- lm(sim2_data$Y ~ ., data = sim2_data$X %>% data.frame())

    ## get estimated beta1 from the "true" model
    truemodel_beta1 <- summary(true_model_res)$coefficients[2, 1]

    ## runs correlated model
    corr_model_res <- lm(sim2_data$Y ~ ., data = sim2_data$X_w_correlated_variates %>% data.frame())
```

```
    ## get estimated beta1 from the colinear model
    colinearmodel_beta1 <- summary(corr_model_res)$coefficients[2, 1]

    return(data.frame(
        n_correlated_covars=n_correlated_covars,
        corr=corr,
        true_beta1=true_beta1,
        truemodel_beta1=truemodel_beta1,
        colinearmodel_beta1=colinearmodel_beta1))
}
```

Part II: Omitted Variables Bias

Recall that omitting confounders from our model will introduce bias to the coefficient estimates. In particular, a confounder:

- Is a determinant of the dependent variable,
- Is correlated with the independent variable of interest.

Remember this function to estimate β_1 (with random seed=5) with covariates that are correlated with $\vec{x}_1$:

```
N <- 250
P <- 20

modelCorrData(n=N, p=P, n_correlated_covars = 1, corr = 0.6, seed = 5)
```

```
##   n_correlated_covars corr true_beta1 truemodel_beta1 colinearmodel_beta1
## 1                   1  0.6        0.5           0.438               0.363
```

Q1: Complete this function to re-run `modelCorrData()` over a vector of seeds and report the average empirical bias.

NOTE: We will use the convention for empirical bias: $bias(\hat{\beta}) = \hat{\beta} - \beta$

```
biasOfCorrData <- function(n=N, p=P, n_correlated_covars = 1, corr = 0.6, seed_vec = 1:100) {
    ## initialize bias:
    truemodel_total_bias = 0
    colinearmodel_total_bias = 0

    for(s in seed_vec) {
        results <- modelCorrData(
            n=N, p=P,
            n_correlated_covars = n_correlated_covars,
            corr = corr, seed = s)

        ## TODO: Compute the bias under the "true model" and the colinear model
        truemodel_bias <- results$truemodel_beta1[1] - results$true_beta1[1]
        colinearmodel_bias <- results$colinearmodel_beta[1] - results$true_beta1[1]
```

```
        ## add to total bias counter
        truemodel_total_bias <- truemodel_total_bias + truemodel_bias
        colinearmodel_total_bias <- colinearmodel_total_bias + colinearmodel_bias
    }

    ## TODO: average the total bias numbers, then take their absolute value :
    truemodel_abs_mean_bias <- abs(truemodel_total_bias/length(seed_vec))
    colinearmodel_abs_mean_bias <- abs(colinearmodel_total_bias/length(seed_vec))



    return(data.frame(n_correlated_covars = n_correlated_covars,
                      corr = corr,
                      truemodel_abs_mean_bias,
                      colinearmodel_abs_mean_bias))
}
```

Once finished with above, uncomment and test out the function:

```
biasOfCorrData(n=N, p=P, n_correlated_covars = 1, corr = 0.6, seed_vec = 1:100)
```

```
##   n_correlated_covars corr truemodel_abs_mean_bias colinearmodel_abs_mean_bias
## 1                   1  0.6                 0.00906                      0.0136
```

```
biasOfCorrData(n=N, p=P, n_correlated_covars = 1, corr = 0.9, seed_vec = 1:100)
```

```
##   n_correlated_covars corr truemodel_abs_mean_bias colinearmodel_abs_mean_bias
## 1                   1  0.9                 0.00721                      0.0244
```

```
biasOfCorrData(n=N, p=P, n_correlated_covars = 50, corr = 0.9, seed_vec = 1:100)
```

```
##   n_correlated_covars corr truemodel_abs_mean_bias colinearmodel_abs_mean_bias
## 1                  50  0.9                 0.00788                      0.0259
```

Q2: The empirical bias is slightly increasing as we exacerbate the collinearity issue, even though the collinear variables have no explicity dependence to $\vec{y}$. Why do you think that is?

YOUR ANSWER HERE: The variance should be increasing as we exacerbate the collinearity issue, meaning the bias will go to 0 as we increase the length of our seed vector.

Based on the answer above, we should expect the bias to decrease if we use more simulation runs:

```
biasOfCorrData(n=N, p=P, n_correlated_covars = 1, corr = 0.9, seed_vec = 1:1000)
```

```
##   n_correlated_covars corr truemodel_abs_mean_bias colinearmodel_abs_mean_bias
## 1                   1  0.9                 0.00265                     0.000958
```

Indeed, the average bias has drastically reduced with 10-times the number of simulation runs.

Q3: Complete the following function to make $\vec{x}_1$ correlated with $\vec{x}_2$, then remove $\vec{x}_2$ from the "omitted variable" dataset.

```
omittedSimulation <- function(n, p, corr = 0.6) {
    ## generate 2 correlated variables
    correlated_variates <-
        faux::rnorm_multi(n = n, vars = 2, r = corr)

    ## main DGP:
    X <- generateX(n, p)

    ## replace first two non-intercept covariates with correlated_variate:
    X[,2] <- correlated_variates[,1]
    X[,3] <- correlated_variates[,2]

    beta <- generateBeta(p)
    Y <- generateY(X, beta)

    ## TODO: remove the second non-intercept covariate from the omitted data:
    X_w_omitted_variable <- X[,c(1:2, 4:ncol(X))]



    return(list(X=X, X_w_omitted_variable=X_w_omitted_variable, beta=beta, Y=Y))
}
```

The following functions will run regression pipeline for the omitted data and report results, very similar to multicollinearity functions (please briefly review):

```
modelOmittedData <- function(n, p, corr = 0.6, seed = 5) {
    ## we will set seed to make sure the results are reproducible
    set.seed(seed)

    sim_data <- omittedSimulation(n, p, corr = corr)

    ## get true beta1
    true_beta1 <- sim_data$beta[2]

    ## runs model on the "true" data matrix
    true_model_res <- lm(sim_data$Y ~ ., data = sim_data$X %>% data.frame())

    ## get estimated beta1 from the "true" model
    truemodel_beta1 <- summary(true_model_res)$coefficients[2, 1]

    ## runs correlated model
    omitted_model_res <- lm(sim_data$Y ~ ., data = sim_data$X_w_omitted_variable %>% data.frame())

    ## get estimated beta1 from the colinear model
    omittedmodel_beta1 <- summary(omitted_model_res)$coefficients[2, 1]

    return(data.frame(
        corr=corr,
```

```
        true_beta1=true_beta1,
        truemodel_beta1=truemodel_beta1,
        omittedmodel_beta1=omittedmodel_beta1))
}

biasOfOmittedData <- function(n=N, p=P, corr = 0.6, seed_vec = 1:100) {
    ## initialize bias:
    truemodel_total_bias = 0
    omittedmodel_total_bias = 0

    for(s in seed_vec) {
        results <- modelOmittedData(
            n=N, p=P,
            corr = corr, seed = s)

        ## Compute the bias under the "true model" and the colinear model
        truemodel_bias <- results$truemodel_beta1[1] - results$true_beta1[1]
        omittedmodel_bias <- results$omittedmodel_beta1[1] - results$true_beta1[1]

        ## add to total bias counter
        truemodel_total_bias <- truemodel_total_bias + truemodel_bias
        omittedmodel_total_bias <- omittedmodel_total_bias + omittedmodel_bias
    }


    truemodel_abs_mean_bias <- abs(truemodel_total_bias / length(seed_vec))
    omittedmodel_abs_mean_bias <- abs(omittedmodel_total_bias / length(seed_vec))

    return(data.frame(corr = corr,
                      truemodel_abs_mean_bias,
                      omittedmodel_abs_mean_bias))
}
```

Recall in these functions, we are examining estimation with respect to the first covariate, which has value 0.5. Now, we are correlating the second **true** covariate with the first, and then omitting it from the data.

Q4: Use function `biasOfOmittedData()` to test the impact of omitting the second covariate when it has *correlation* $= 0.4$ with the first covariate, then with *correlation* $= 0.9$. What do you see? What happens if you increase the number of runs to seeds 1 to 1,000?

```
biasOfOmittedData(n=N, p=P, corr = 0.4, seed_vec = 1:100)
```

```
##   corr truemodel_abs_mean_bias omittedmodel_abs_mean_bias
## 1  0.4                 0.0109                       0.183
```

```
biasOfOmittedData(n=N, p=P, corr = 0.9, seed_vec = 1:100)
```

```
##   corr truemodel_abs_mean_bias omittedmodel_abs_mean_bias
## 1  0.9                 0.0232                       0.406
```

```
biasOfOmittedData(n=N, p=P, corr = 0.4, seed_vec = 1:1000)
```

```
##   corr truemodel_abs_mean_bias omittedmodel_abs_mean_bias
## 1  0.4                0.000532                      0.178
```

```
biasOfOmittedData(n=N, p=P, corr = 0.9, seed_vec = 1:1000)
```

```
##   corr truemodel_abs_mean_bias omittedmodel_abs_mean_bias
## 1  0.9                 0.00131                      0.401
```

YOUR ANSWER HERE: Increasing the correlation of the confounder to a covariate does increase the bias of the omitted confounder model. However, increasing the number of simulated trials does not change the bias.

Q5: Will increasing sample size alleviate the issues of multicollinearity? How about omitted variables bias? Explain.

YOUR ANSWER HERE: The variance of β_1 of the omitted variable model decreases while the bias stays the same.

Part III: Validation set performance.

In this last section, we will briefly walk through the mechanics of creating a holdout data set, and computing prediction performance MSE on this holdout (validation data).

First, lets generate some data with collinearity:

```
collinear_data <- collinearitySimulation(
    n=N, p=P, n_correlated_covars = 30, corr = 0.7)

collinear_data$X_w_correlated_variates <- collinear_data$X_w_correlated_variates %>% as.matrix()

print(dim(collinear_data$X_w_correlated_variates))
```

```
## [1] 250  51
```

Q6: Complete the following code to holdout the first 30 rows of each simulated data object.

```
y_holdout <- collinear_data$Y[1:30, ]
X_holdout <- collinear_data$X[1:30, ]
X_collinearity_holdout <- collinear_data$X_w_correlated_variates[1:30, ]

## TODO: create the training datasets with all the other rows of the simulation
y_train <- collinear_data$Y[31:N,]
X_train <- collinear_data$X[31:N,]
X_collinearity_train <- collinear_data$X_w_correlated_variates[31:N,]
```

Please uncomment code out once the above exercise is complete.

First, we will put the above objects into data.frames:

```
true_train_df <- X_train %>% data.frame() %>% mutate(y=y_train)
collinear_train_df <- X_collinearity_train %>% data.frame() %>% mutate(y=y_train)

true_holdout_df <- X_holdout %>% data.frame() %>% mutate(y=y_holdout)
collinear_holdout_df <- X_collinearity_holdout %>% data.frame() %>% mutate(y=y_holdout)
```

Now we can run a model on **only** the training data:

```
true_lm <- lm(y ~ 0 + ., data = true_train_df)
collinear_lm <- lm(y ~ 0 + ., data = collinear_train_df)
```

Q7: Generate predictions for `y_holdout` using both of the above models.

```
## YOUR CODE HERE:
true_lm_holdout_preds <- predict(true_lm, true_holdout_df)
collinear_lm_holdout_preds <- predict(collinear_lm, collinear_holdout_df)
```

Q8: Compute and print the mean square prediction error for the "true" model and the "collinear" model. Why do you think the performance for the true model is better?

```
### YOUR CODE HERE:
mse_true_model = mean((true_lm_holdout_preds - true_holdout_df$y)^2)
mse_collinear_model = mean((collinear_lm_holdout_preds - collinear_holdout_df$y)^2)

print(mse_true_model)
```

```
## [1] 1.24
```

```
print(mse_collinear_model)
```

```
## [1] 1.25
```

YOUR ANSWER HERE: Collinear model overfits the response to our collinear features (have no deterministic effect on our respose variable), making the model not generalizable for out validation dataset.

You finished!